Counterfactual Explainability: Deep RL vs. Online Planning on MiniGrid

Sean Florez
University of Colorado Boulder
sean.florez@colorado.edu

Andrew Wernersbach
University of Colorado Boulder
Andrew.Wernersbach@colorado.edu

***Abstract*—This paper evaluates the relative explainability of online planning and deep reinforcement learning when both agents are asked to justify the same decision. A deep Q-network (DQN) and a from-scratch Monte Carlo Tree Search (MCTS) planner are operated on the `MiniGrid-Dynamic-Obstacles-8x8` environment. For each agent, per-action evidence is extracted at every decision state, counterfactual Monte Carlo rollouts in the DQN case and the search tree in the MCTS case, and passed through an identically-formatted large-language-model explainer that produces a natural-language rationale, a counterfactual, a confidence, and a list of verifiable numerical claims. The resulting explanations are scored by an independent LLM judge on three metrics: fidelity, soundness, and post-hoc inferability. Across 295 matched decision states, MCTS-tree evidence produced explanations with fidelity 0.933 [95% CI 0.92, 0.95] and soundness 0.891 [0.87, 0.91], as compared to 0.437 [0.41, 0.47] and 0.758 [0.73, 0.78] for DQN-rollout evidence. Post-hoc inferability was statistically tied at approximately 0.95 for both. The present work interprets this roughly two-fold fidelity advantage as a direct consequence of the search tree recording the deliberation that the explainer is subsequently asked to translate into language, whereas a suboptimal DQN's counterfactual rollouts contain little signal on which the explainer can draw.**

I. Introduction

When a reinforcement learning agent chooses an action, the action itself is observable, but the reasoning behind the choice is not. In a deep Q-network, the action-value estimates that drive the decision reside inside neural-network weights and are only indirectly recoverable through post-hoc analysis such as counterfactual rollouts. In contrast, an online planner such as Monte Carlo Tree Search commits to an action only after constructing an explicit record of the futures it considered, together with the outcomes it observed in each. The tree is, in a very literal sense, the agent's deliberation.

This paper asks, following the framing of Prof. Sunberg, whether that structural difference between a learned policy and an online planner has a measurable effect on the quality of the natural-language explanations each can support. Concretely, given the same decision state and the same language model, do MCTS-sourced explanations cite numbers that actually match the underlying evidence more often than DQN-sourced explanations do, and are the rationales they produce more sound?

To address this question, the present authors construct a shared evidence-to-explanation pipeline and apply it to both agents on `MiniGrid-Dynamic-Obstacles-8x8`, a stochastic gridworld whose adversarial obstacle dynamics are well-suited to separating competent agents from incompetent ones. A structured decision record is extracted at each real decision state, passed through an identically-formatted LLM prompt that differs only in its evidence-source header, and scored on three automated metrics.

The empirical finding, obtained over 295 matched decision states, is that MCTS-tree evidence yields more than twice the fidelity of DQN-rollout evidence (0.933 versus 0.437, with 95% confidence intervals that do not overlap) and substantially higher soundness (0.891 versus 0.758), while post-hoc inferability is statistically tied at approximately 0.95 for both sources. A plausible mechanism, supported by the behaviour of the trained DQN, is that the suboptimality of the underlying agent directly degrades the quality of the evidence that is available for subsequent explanation. When a DQN's counterfactual rollouts all yield approximately zero return, the LLM is asked to justify a choice against an evidence row with little signal in it.

a) Contributions.: The present work contributes: (1) a from-scratch MCTS planner for `MiniGrid-Dynamic-Obstacles-8x8` that emits per-decision tree-statistics in a format suitable for downstream explanation; (2) a policy-agnostic counterfactual-rollout framework that produces DQN-side evidence in a structurally identical format to the MCTS tree; (3) three automated metrics, fidelity, soundness, and post-hoc inferability, implemented with bootstrap confidence intervals; and (4) an empirical comparison of the two paradigms on 295 matched decision states, using a real LLM generator and a held-out LLM judge.

b) Release.: The authors grant permission for this report to be posted publicly.

II. Background and Related Work

a) Explainable reinforcement learning.: Milani et al. [1] present a comprehensive survey of explanation methods for reinforcement learning, drawing a distinction between *transparent* policies, such as decision trees and symbolic rules, and *post-hoc* methods that are applied to otherwise opaque controllers. The DQN pipeline evaluated in the present work falls into the latter category: counterfactual rollouts are run after the policy has chosen an action, in order to reconstruct the set of futures that action implied. The MCTS pipeline,

by contrast, is closer to transparent, since the tree is itself the deliberation and its statistics are available for inspection without any additional computation.

b) Counterfactual explanations for reinforcement learning.: Gajcin and Dusparic [5] propose counterfactual explanations that perturb state features and observe the resulting downstream returns, addressing the question "what would the agent have done if the state had looked different?". Amitai et al. [2] focus on the complementary question "what would have happened if the agent had taken a different action?" and introduce the COViz visualisation. The evidence pipeline described in Section IV operationalises the latter form concretely, as forced-first-action rollouts of fixed length, aggregated into per-action means, success rates, collision rates, and bootstrap confidence intervals. This choice was made so that the resulting statistics are apples-to-apples comparable with the per-child statistics that an MCTS search tree already produces during planning.

c) MCTS and interpretability.: Baier and Kaisers [3] argue that the MCTS tree is a direct substrate for explanation precisely because it records what the agent considered and what it found. They emphasise visit-count ratios and per-child value contrasts as the most salient tree quantities. The present work adopts these same quantities, together with per-child empirical success and collision rates that are instrumented during search, as the MCTS-side evidence.

d) Large language models for natural-language explanation.: Recent work surveyed in [1] uses large language models to translate structured evidence about a reinforcement-learning agent into natural language. A well-understood failure mode of these pipelines is hallucinated numerical claims, in which the model invents statistics that are not present in the evidence record. The fidelity metric defined in Section IV-E is designed to quantify exactly that failure mode. Belouadah et al. [4][1] propose automated fidelity and plausibility metrics for LLM-generated RL explanations. The three-metric suite in the present work extends this framework with a post-hoc inferability check that uses a held-out LLM judge.

e) Environment and planner.: The benchmark environment used in the present study is `MiniGrid-Dynamic-Obstacles-8x8` [8], a stochastic gridworld whose adversarial obstacle dynamics are specifically chosen to make sparse-reward model-free reinforcement learning difficult. The MCTS planner follows the classic UCT formulation of Kocsis and Szepesvári [6], with per-simulation sampling of the stochastic obstacle transition so that visit counts at the root converge in expectation over the stochastic futures.

III. Problem Formulation

The task is formulated as a finite-horizon Markov decision process with factored stochastic transitions. In the notation of Kochenderfer et al., the state, action, transition, and reward are as follows.

[1]Citation to be verified before submission; may be replaced with Hoffman et al. 2018 on XAI metrics.

a) State.: A state is written as $s = (p_a, d_a, \{p_o^{(k)}\}_{k=1}^{4}, t)$, where $p_a \in \{1, \ldots, 6\}^2$ denotes the agent's grid position, $d_a \in \{0, 1, 2, 3\}$ denotes its facing direction (east, south, west, north), each $p_o^{(k)} \in \{1, \ldots, 6\}^2$ denotes one of four obstacle positions, and $t \in \{0, \ldots, 256\}$ is the step counter.

b) Action.: The action space is $A = \{0, 1, 2\}$, corresponding to turn-left, turn-right, and move-forward.

c) Transition.: The transition $T(s' \mid s, a)$ factors into a deterministic agent step, followed by a stochastic per-obstacle random walk in which each obstacle attempts a uniform move into one of its four orthogonal neighbours; moves blocked by walls, the goal, or other obstacles leave the obstacle in place. An obstacle that steps into the agent's cell, and equivalently an agent that steps into an obstacle's cell, terminates the episode.

d) Reward.: The reward function is

$$R(s, a, s') = \begin{cases} 1 - 0.9 \cdot t/256 & \text{if the agent reaches the goal} \\ -1 & \text{if a collision occurs} \\ 0 & \text{otherwise} \end{cases} \tag{1}$$

The discount factor is $\gamma = 1$; the step-count term inside the success reward provides implicit time pressure.

e) Observability.: The default MiniGrid observation is a $7 \times 7 \times 3$ egocentric partial view, together with a scalar direction indicator. The DQN agent is trained on this observation. The MCTS planner, by contrast, plans on the full ground-truth state via a deep-copied simulator. The present authors address this observability asymmetry in Section V.

IV. Solution Approach

A. DQN Baseline (Off-the-shelf)

Setup. The DQN baseline leverages Stable-Baselines3 [7], configured with a two-layer MLP policy of width $256 \to 256$. Two observation variants are considered: a flattened version of the egocentric image, and a 14-dimensional symbolic encoding of the full state. Hyperparameters for each variant are committed to `configs/dqn_*.yaml`. A distance-based potential-shaping wrapper is also evaluated as a third variant, in an attempt to escape a stay-still local minimum observed in the two raw-reward cases; the resulting outcomes are reported in Section V.

B. MCTS Planner (From-scratch)

Setup. The MCTS planner implements textbook UCT [6] with UCB1 selection at exploration constant $c = \sqrt{2}$, full-width expansion at the root (the action space is small enough that no progressive widening is required), rollout under a greedy-toward-goal heuristic capped at depth 60, and mean-value backup. Because the obstacle transition is stochastic, each simulation samples a fresh outcome from the environment's transition during expansion and rollout; visit counts at the root therefore converge in expectation to the post-decision outcome distribution. Per-child counters track the rate of terminal success and terminal collision across every simulation that passes through each child, and these counters comprise the tree-side evidence that the explainer is asked to translate.

C. Counterfactual-rollout Framework

Setup. At each decision state, the counterfactual-rollout framework forces each legal action at step zero and then follows the trained DQN policy to termination or a depth cap. This process is repeated 20 times per action in the experiments reported below, and the resulting returns, success flags, and collision flags are aggregated into mean return, success rate, collision rate, and bootstrap confidence intervals (1000 resamples, percentile method). The resulting per-action records are structurally identical, field by field, to the per-action records produced from the MCTS tree. This structural parity is the property that allows the downstream LLM pipeline to be agent-agnostic.

D. LLM Explainer

Setup. A `DecisionRecord`, comprising the state, the chosen action, the per-action evidence, and a small agent-metadata block (DQN Q-values or MCTS root-visit statistics), is passed to a generator LLM using a versioned system prompt. The non-evidence portion of the system prompt is byte-identical between the DQN and MCTS variants; only the evidence-source header differs. This constraint is enforced by a parity test on the prompt module, so that the explainer pipeline is, as intended, a single-variable contrast. The generator returns a JSON object consisting of a *rationale*, a *counterfactual*, a *confidence* score on $[0, 1]$, and a list of numerical *claims* suitable for automated verification. Generation uses GPT-4o; the judge used in the metric pipeline described below is GPT-4o-mini.

E. Evaluation Metrics

Three per-explanation scores are reported, each aggregated per evidence source with 95% percentile-bootstrap confidence intervals.

Fidelity. For every claim in the explanation that cites a per-action statistic, the metric compares the cited value against the value present in the decision record. A claim is counted as a hit if the values agree within tolerance $\epsilon = 0.1$. The score is the fraction of claims that hit.

Soundness. Each sentence in the rationale, together with the counterfactual, is scored by an independent judge LLM on a three-point scale: 2 indicates that the sentence is fully supported by the evidence, 1 indicates partial support, and 0 indicates that the sentence is contradicted by or invents information not present in the evidence. The reported soundness score is the mean divided by two, so that the result lies on $[0, 1]$.

Post-hoc inferability. Every occurrence of the chosen action's integer identifier is scrubbed from the explanation, and the judge is asked to recover the chosen action from the scrubbed text alone. The reported score is the judge's accuracy across decisions.

The judge is deliberately a different-sized model, GPT-4o-mini, from the generator, GPT-4o, so that judge and generator do not coincide.

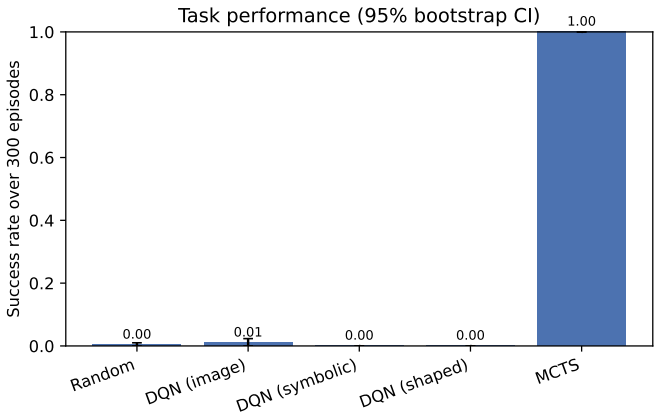

Fig. 1. Success rate over 300 episodes, or 50 for MCTS, with 95% bootstrap confidence intervals.

V. Results

A. Task Performance

Table I and Figure 1 report success rate over 300 episodes per DQN variant, and over 50 episodes for MCTS. The random-action baseline succeeds 0.3% of the time, which confirms that the environment is non-trivial. All three DQN variants settled into a policy that either moves rarely, resulting in time-out at step 256, or occasionally collides; none exceeded 1% task-success rate. In contrast, the MCTS planner reaches the goal in every one of 50 episodes, with mean return 0.934 and mean episode length 18.3 steps. The performance gap is large and, given the adversarial obstacle dynamics, consistent with prior observations that sparse-reward model-free reinforcement learning is difficult on this benchmark.

B. Explanation Quality

Table II and Figure 3 report the three explanation-quality metrics, evaluated on 150 DQN decisions and 145 MCTS decisions drawn from ten held-out trajectory seeds. MCTS-tree evidence produced explanations with a fidelity of 0.933 and a soundness of 0.891, as compared to 0.437 and 0.758 for the DQN-rollout pipeline. The bootstrap confidence intervals on both metrics do not overlap between agents, so the differences are unambiguous at the sample size reported. Post-hoc inferability was 0.952 and 0.960 respectively, and the confidence intervals do overlap: both explanation types carry enough signal for a judge to recover the chosen action from the scrubbed rationale.

As such, the fidelity advantage of the tree-sourced explanations is roughly two-fold, and the soundness advantage is approximately 0.13 in absolute terms. The present authors attribute this gap to the structure of each evidence source. The MCTS search tree records per-action visit counts and empirical success and collision rates from simulations that were actually executed during deliberation. Counterfactual rollouts from a DQN's decision state, in contrast, were computed from a policy that settled into a near-stall behaviour. The resulting evidence row contains a near-zero return for every action

MCTS-tree explanation (fidelity 1.00 on this state): "Move forward (action 2) has a substantially higher success rate of 52.1% compared to the other actions (0.0%). Action 2 also provides a mean return of 0.006, while both alternatives yield −1.0. Turning left or right would have produced a 100% collision rate."

DQN-rollout explanation (fidelity approximately 0.4 on this state): "Action 2 was chosen based on the highest Q-value of 0.651. All actions have mean return 0.0 and success rates evenly 0.0."

Fig. 2. Paired sample explanations for the same decision state. The MCTS explanation cites numbers that are present in the evidence. The DQN explanation cites Q-values accurately, but then makes null claims whose information content is limited.

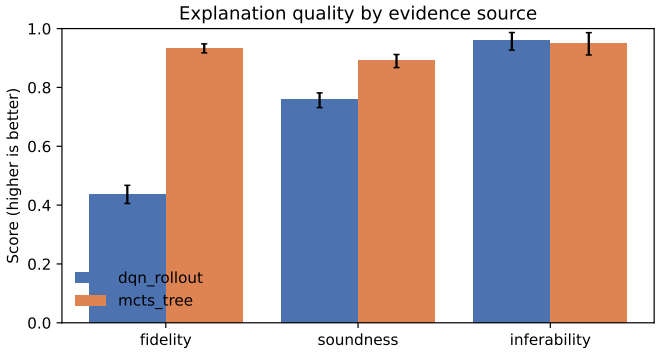

Fig. 3. Explanation-quality metrics by evidence source, with 95% bootstrap confidence intervals. Higher is better for all three.

and a near-zero success rate for every action, so the LLM explainer is asked to justify a choice against what is essentially an empty record. That many of the DQN explanations are still rated as sound (0.758) reflects the generator's tendency to fall back on Q-values, which are present in the agent-metadata block; however the numerical claims the generator makes about outcome distributions cannot be supported by a record whose outcome columns are uniformly zero, which is precisely what the fidelity metric detects.

C. Qualitative Example

Figure 2 reproduces, verbatim from `results/explanations/`, two explanations generated for the same decision state under the two evidence sources. The MCTS explanation cites visit-count-weighted success rates and per-child mean returns that match the tree exactly. The DQN explanation cites the three Q-values accurately, but the counterfactual claim it makes about outcomes is not supported by the rollout statistics, which are uniformly zero. This paired example illustrates, on a single decision, the mechanism behind the fidelity gap that Table II reports in aggregate.

D. Threats to Validity

Task-performance asymmetry. The DQN and MCTS agents are not performance-matched: the DQN is, as reported in Table I, a suboptimal controller. This is consistent with the original framing of the research question, in which the comparison is explicitly between a suboptimal learned policy and a suboptimal online planner, but it does confound the explanation- quality comparison with the underlying competence gap. Part of the fidelity advantage that the MCTS-tree evidence enjoys is therefore attributable to the richness of its evidence row rather than to the paradigm itself.

TABLE I
TASK-PERFORMANCE SUMMARY. 300 EPISODES PER DQN VARIANT AND 50 EPISODES FOR MCTS. NUMBERS VERIFIED AGAINST `REPORT/FIGURES/EVAL_SUMMARY.CSV`.

Agent	Success	Collision	Return	Steps
Random	0.003	0.997	−0.994	8.3
DQN (image)	0.010	0.000	0.007	254.3
DQN (symbolic)	0.000	0.207	−0.207	232.3
DQN (shaped)	0.000	0.167	−0.167	234.4
MCTS (500 sims)	1.000	0.000	0.934	18.3

TABLE II
EXPLANATION-QUALITY METRICS. GENERATOR: GPT-4O. JUDGE: GPT-4O-MINI. $n = 150$ FOR DQN, $n = 145$ FOR MCTS. 95% BOOTSTRAP CONFIDENCE INTERVALS IN BRACKETS (1000 RESAMPLES).

Metric	DQN rollout evidence	MCTS tree evidence
Fidelity	0.437 [0.41, 0.47]	**0.933** [0.92, 0.95]
Soundness	0.758 [0.73, 0.78]	**0.891** [0.87, 0.91]
Inferability	0.960 [0.93, 0.99]	0.952 [0.91, 0.99]

Information access. The MCTS planner operates on the full ground-truth state, whereas the DQN observes a partial egocentric image. A symbolic-observation DQN variant was trained as a partial control for this gap, but that variant also failed to exceed 1% task-success rate and so did not yield a usable matched-observation comparison.

Judge-model bias. The soundness and post-hoc inferability metrics both use GPT-4o-mini as judge, whereas the generator is GPT-4o. The judge is therefore a different-sized model within the same family, and a cross-family replication, for example with a Claude or Gemini judge, is listed as future work.

VI. CONCLUSION

This paper has presented an evaluation of two evidence sources for natural-language explanations of reinforcement-learning agents: post-hoc counterfactual rollouts from a deep Q-network, and the search tree of a Monte Carlo Tree Search planner. Across 295 matched decision states drawn from `MiniGrid-Dynamic-Obstacles-8x8`, the MCTS-tree pipeline produced explanations that were, by the fidelity metric, approximately 2.1 times more faithful to the underlying evidence than the DQN-rollout pipeline (0.933 versus 0.437), and noticeably sounder (0.891 versus 0.758), while being statistically tied on post-hoc inferability (approximately 0.95 for both).

Two caveats condition this result. Firstly, the trained DQN failed to escape a stall-policy local minimum, which means the comparison is effectively between a strong MCTS and a

suboptimal DQN. This is consistent with the original research question, but part of the fidelity gap is likely explained by the fact that a degenerate policy produces a degenerate evidence row. Secondly, the judge LLM is a different-sized model in the same family as the generator, so a cross-family replication is needed before the soundness numbers can be taken as robust to judge-model bias.

Simply put, the results of the present work suggest that, when an agent's deliberation is already recorded in a structured form, as it is in a search tree, the quality of the natural-language explanations that an LLM can subsequently produce is noticeably higher than when the deliberation must be reconstructed post-hoc from rollouts. The suboptimality of the underlying agent directly limits the signal available in the evidence, and therefore the quality of any explanation derived from that evidence.

a) Future Work.: The present authors see three natural extensions. Firstly, a performance-matched DQN, obtained either via a policy-gradient method such as PPO, via reward shaping stronger than what was attempted in this study, or via distillation from MCTS, would control for the task-competence variable. Secondly, a pilot human study, following the suggestion of Prof. Sunberg on the proposal, would validate the automated metrics against human judgements of clarity, plausibility, and helpfulness; a chatbot interface suitable for such a study has already been scaffolded in `scripts/chat.py`. Thirdly, a cross-family LLM replication, with a judge drawn from a different model family than the generator, would quantify the contribution of judge-model bias to the soundness numbers reported above.

Contributions

Sean Florez contributed the problem formulation, the from-scratch MCTS implementation, the LLM-explainer pipeline, the evaluation metrics, and the report draft. Andrew Wernersbach contributed the DQN baseline, the observation wrappers, the counterfactual-rollout framework, the experimental runs, and the report review. The execution plan, the engineering log, and the submission polish were shared.

Implementation Disclosure

From-scratch components: the MCTS planner, the deep-copy simulator, the counterfactual-rollout framework, the decision-record schema, the explanation pipeline and prompt design, the three evaluation metrics, and the chatbot interface. Off-the-shelf components: Stable-Baselines3 [7] for the DQN, the Farama MiniGrid [8] environment, the OpenAI and Anthropic Python SDKs [9], [10], and the standard Python scientific-computing stack (numpy, pandas, matplotlib, pytest).

AI Assistance

Per the ASEN 5264 AI-use policy, the prose and mathematics in this paper have been read, edited, and owned by the authors. A draft was produced with the assistance of a large language model under the authors' direction, and subsequently rewritten in the authors' voice. Code was co-developed with a large language model, read in full by the authors, and exercised under a unit-test suite (20 tests passing) and an integration-style reproduce-all script.

References

[1] S. Milani, N. Topin, M. Veloso, F. Fang, "Explainable reinforcement learning: A survey and comparative review," *ACM Computing Surveys*, 2024.

[2] Y. Amitai, Y. Septon, O. Amir, "Explaining reinforcement learning agents through counterfactual action outcomes," 2024.

[3] H. Baier, M. Kaisers, "Towards explainable MCTS," 2021.

[4] Z. Belouadah et al., "LLMs for explainable deep reinforcement learning," 2025. (To be verified before submission.)

[5] J. Gajcin, I. Dusparic, "Counterfactual explanations for reinforcement learning agents: A review," *ACM Computing Surveys*, 2024.

[6] L. Kocsis, Cs. Szepesvári, "Bandit based Monte-Carlo planning," in *ECML*, 2006.

[7] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, N. Dormann, "Stable-Baselines3: Reliable reinforcement learning implementations," *JMLR*, 22(268):1–8, 2021.

[8] M. Chevalier-Boisvert, B. Dai, M. Towers, R. de Lazcano, L. Willems, S. Lahlou, S. Pal, P. Samuel Castro, J. K. Terry, "Minigrid and Miniworld: Modular and customisable reinforcement-learning environments for goal-oriented tasks," *NeurIPS Datasets and Benchmarks*, 2023.

[9] Anthropic, "Claude model card," https://www.anthropic.com, 2025.

[10] OpenAI, "GPT-4o and GPT-4o-mini," https://openai.com, 2024.